Cancellation coverage matrix — operation-scoped cancellation (`6e09ced`)

Candidate	**`6e09ced4bf42d036c257cb35772335600215ff30`** on `fix/operation-scoped-cancellation`
Lineage	`f764c95` (B1 recovery scope + B6 killable subprocess) → `b317833` (B2/B3/B7 production-boundary tests + B7 test seam) → `0d05556` (recovery window = real remaining fetch-phase budget) → `782f99d` (bounded in-flight browser interruption, PDF path) → `e4894ae` (obsolete-work settlement for coalesced callers) → `45cc661` (real-IPC proc-admission withdrawal test + explicit unattempted-URL outcomes) → `658a17c` (deadline threading into browser nav timeouts via `remainingMs`) → **`6e09ced`** (bounded in-flight interruption wired into the `fetchWithBrowser` HTML path + abort-aware settle waits)
Base (production)	`origin/main` = `8c03b07` (untouched)
Purpose	Honest requirement→test→assertion map for every cancellation path, stating for each case **what level of realism** the test exercises (real Chromium child / real isolated subprocess / real OS fork+IPC / real Postgres / broker unit / in-process unit) and **what it does not prove**.

This document answers reviewer point 2. It is deliberately conservative: where a test proves *ordering* or *admission-gate* behaviour rather than *mid-flight interruption*, it says so, and the open gaps are listed explicitly in the final section rather than folded into a "covered" row.

Realism legend

Level	Meaning
R — real Chromium child	A real isolated job child (`runPipelineIsolated` → `job-child.ts`) drives the **production** browser call sites against a local HTTP server, with a real Chromium process observed via `/proc`.
R(page) — real Chromium page	A real Chromium page (`PUPPETEER_EXECUTABLE_PATH`) is held mid-operation and interrupted; wall-clock bound asserted against the production interrupt helper.

S — real isolated subprocess	A real OS child process (pdf-parse worker via `runProcessCollect`/`extractViaPdfParseSubprocess`) is spawned and killed; the test observes real SIGKILL / reaping.
I — real fork + IPC	A real OS child process is `fork`ed and driven over the production IPC channel (`job-child.ts` → `isolation-ipc.ts` → parent broker); admission withdrawal observed across the process boundary.
D — real Postgres	The assertion is made against the real `documents`/`companies` tables in the integration Postgres (`ciq-int-pg`), through the production storage functions.
U(broker)	The real admission broker / in-flight map / scope machinery is exercised in-process (no network, no Chromium), driving the actual production functions (`acquireProcSlot`, `processDocument`, `runCompanyPdfRecovery`).
U(unit)	Pure in-process unit assertion over the scope/deadline primitives.

“Production code path” means the assertion runs through the same function production calls — not a re-implementation in the test.

---

1. Deadline / scope propagation (B1 — company PDF recovery)

Boundary: `runCompanyPdfRecovery` (pipeline.ts), called in production (default `fetchFn` = real `fetchIssuerPdfsWithPrimedSession`).

#	Requirement	Test :: subtest	
1.1	One scope is armed and **passed to every fetch call** across all origins	`company-pdf-recovery-boundary.test.ts` :: B1-1	identical `sco` fetch bounda
1.2	Recovery budget is **capped by `min(PDF_RECOVERY_MAX_BUDGET_MS, remainingMs)`**	B1-2	armed deadl `remainingMs`
1.3	**No fetch is dispatched after the deadline**; the untried tail stays retryable	B1-3	origins after `fetchFn`; tail (retryable)
1.4	Attempt budget caps total attempts **across** origins (company-wide, not per-origin)	B1-4	total fetch a summed acr
1.5	Healthy path: all origins fetched, nothing filtered	B1-5	every origin abort
1.6	Default `fetchFn` routes through the **real admission boundary** and withdraws at the company deadline	B1-6	with the pro queued wait company de
1.7	**Production recovery window = the real remaining fetch-phase budget** (not the 600 000	B1-7	`remainingFet` returns the l clamped to a

	ms constant)	constant

Mutation proof (B1-1): removing scope propagation (`fetchFn(origin, slice, { outcomes, scope }) → { outcomes }`) makes **B1-1 fail** (`AssertionError: a scope was passed to the fetch boundary (was DEAD/omitted before B1)`); B1-2/B1-3 cascade. Reverted clean.

Mutation proof (B1-7, this session): replacing the body of `remainingFetchBudgetMs` with `return budgetMs;` (ignore elapsed — `MUTATION_MARKER_B2`) makes **B1-7 fail** (`6 pass / 1 fail`: the window no longer shrinks with elapsed time). Reverted clean; `grep MUTATION_MARKER` empty, tree matches HEAD.

What B1 does NOT prove: B1 operates at the recovery-orchestration boundary, not against a live network fetch. It proves the scope is *constructed, capped, and passed*, the window tracks real elapsed budget, and dispatch stops after the deadline — it does not prove that an HTTP fetch already in flight is torn down mid-transfer (that is the active-interruption boundary, §6 and §7).

---

2. Process-admission cancellation (B2 — `acquireProcSlot`, in-process)

Boundary: `acquireProcSlot` local-broker queue (processor.ts), queued-waiter abort listener.

#	Requirement	Test :: subtest	Key assertion	Lev
2.1	A **queued** proc waiter whose scope expires **withdraws** and is never granted a slot	`cancellable-proc-admission-boundary.test.ts` :: B2-1	waiter settles `withdrawn`, not `hung`; no slot consumed	U(bro
2.2	The next waiter is admitted promptly once the holder releases	B2-2	next waiter granted; `active` returns to expected	U(bro
2.3	Grant/abort race (abort **before** release): waiter withdraws, slot returns cleanly — no leak, no phantom grant	B2-3	`active` drains to 0; no double-grant	U(bro
2.4	A **pre-aborted** scope is refused at	B2-4	throws `OperationAbortedError`;	U(bro

	admission and consumes no slot	`active` unchanged	

Mutation proof (B2): removing the queued-waiter abort listener makes **B2-1 fail** (`'hung' !== 'settled'`), and B2-2/B2-3/B2-4 cascade with `proc slot leaked (active=1)`. Reverted clean.

2b. Process-admission withdrawal across a real process boundary (real fork + IPC)

Boundary: a real isolated child (`runPipelineIsolated` → `job-child.ts`) whose test module calls the **production** `processorSlotBroker.proc.acquire(scope)` → `acquireProcSlot` → `remoteAcquire("proc", …)` over the real IPC channel to the parent broker.

#	Requirement	Test :: subtest	Key assertion	I
2b.1	A child blocked at **proc admission** whose scope aborts **withdraws across IPC** and surfaces an `OperationAbortedError` tagged `proc_admission`	`isolated-proc-admission-boundary.test.ts`	child reports `isOperationAborted === true` with reason `proc_admission`, promptly after abort (≪ grant would arrive)	I

Mutation proof (2b, this session): in `acquireProcSlot`, breaking the abort-error **type** fed to `remoteAcquire` (`() => new OperationAbortedError(…, "proc_admission")` → `() => new Error("MUTATION_MARKER_B5 generic …")`) makes the test **fail fast** (child still withdraws at ~700 ms but surfaces `isOperationAborted: false`). Reverted clean. (A weaker mutation — suppressing the `cancelAcquire` IPC message — did **not** fail the test, because the child exits ~50 ms after `run()` returns and the parent reclaims the queued request on child exit; that masking is documented here rather than hidden, and is why the *error-type* mutation is the load-bearing one.)

This case is strictly stronger in realism than §2's in-process U(broker): the admission queue, the abort, and the withdrawal all cross a real OS process boundary over the production IPC protocol.

---

3. Stalled-extraction cancellation (B3 — `runProcessCollect` subprocess)

Boundary: `runProcessCollect` abort registration, SIGKILL in `onAbort`.

#	Requirement	Test :: subtest	Key assertion	Le
			parent settles **promptly**	

3.1	A **hung extraction child** is SIGKILLed on scope abort; parent stays responsive; slot restored	`stalled-extraction-boundary.test.ts` :: B3	**after abort** (elapsed ≪ child timeout); slot restored	**S**
3.2	`extractViaPdfParseSubprocess` throws **retryable** on an already-aborted scope and takes **no** slot	B3/B2	throws; `active` unchanged	**S**

Mutation proof (B3): disabling the abort registration block makes **B3 fail** with `parent settled promptly after abort (elapsed=60005ms)` — the child runs the **full 60 s timeout** instead of being killed at abort. Reverted clean. This is the clearest active-interruption proof in the subprocess layer: a real child process is really killed at the abort instant.

---

4. In-flight coalescing cancellation + obsolete-work settlement (B7 — `processDocument`)

Boundary: `processDocument` in-flight dedup + liveness guard + coalesced-caller self-settlement (processor.ts).

#	Requirement	Test :: subtest	Key assertion
4.1	Concurrent same-URL callers **coalesce to one** underlying fetch	`inflight-coalescing-boundary.test.ts` :: B7-1	exactly 1 underlying fetch for callers
4.2	A new caller does **NOT** coalesce onto an in-flight entry whose scope has **aborted** — it starts its own attempt	B7-2	2 underlying fetches (aborted reused)
4.3	A caller **coalesced onto** an in-flight op whose **own** scope then aborts **settles to a terminal state** (does not hang waiting on the	B7-3	the coalesced caller's promise own abort via `Promise.race([existing.promis` `callerScope.whenAborted("coal` it never waits past its own dea

		shared op it no longer cares about)		

Mutation proof (B7), three mutants: - Disable coalescing (`if (false && existing …))` → **B7-1 fails** (`2 !== 1`). - Drop the liveness check (ignore `scope.aborted()`) → **B7-2 fails** (`1 !== 2` — the new caller wrongly coalesces onto the aborted entry). - Drop the coalesced-caller race (`Promise.race([existing.promise, callerScope.whenAborted(…)])` → `return existing.promise;` — `MUTATION_MARKER_B4`, this session) → **B7-3 fails** (`2 pass / 1 fail`: the coalesced caller hangs on the obsolete shared op past its own abort). Reverted clean.

All reverted clean. Three independent mutants are needed because the three guards (coalesce, liveness, self-settlement) are independently load-bearing.

---

5. Browser lifecycle close/reap ordering (real Chromium) — `browser-close-order.test.ts`

Boundary: real isolated job children driving **production** browser call sites (`fetchPdfViaBrowser`, `fetchIssuerPdfsWithPrimedSession`), `MAX_CONCURRENT_BROWSER=1`, live Chromium observed via `/proc` at every permit grant/return. Invariant asserted on **every** exit path: **work stopped → Chromium closed → termination confirmed → permit returned**.

#	Requirement	Test :: subtest	Key assertion	Leve
5.1	**Normal op:** marker extracted; Chromium gone before the permit returns	success path	`r.ok`, marker matched, `liveChromium == []` at release	**R**
5.2	**Fetch failure** (connection reset; direct + pdfRecovery): close before release	fetch-failure path	no text yielded; `liveChromium == []` at both releases	**R**
5.3	**Cancellation — admission gates:** a caller refused **after** acquiring the permit, and one refused **before** acquiring, both raise `BrowserUnavailableError: lifecycle cancelled`; permits return with Chromium closed	cancellation path (B_postAcquire, C_preAcquire)	both error strings match; `liveChromium == []` at release	**R**
	Cancellation — in-flight: a fetch already holding the permit at	cancellation	`A_inflight.ok == true`;	

5.4	the cancel instant **completes**, then Chromium closes before the permit returns	path (A_inflight)	`liveChromium == []` at release	**R**
5.5	**Child death:** pipeline deadline → SIGKILL while holding the permit → parent group reap before `releaseAll`	child-death path	job fails; `liveChromium == []` at release; broker drained	**R**

Mutation proof (ordering): documented in the prior browser-lifetime review — releasing the permit *before* closing Chromium makes these tests fail (the `/proc` scan sees a live Chromium at release). Not re-run here (no code in that path changed in this candidate).

What §5 proves and what it does NOT: - It **proves** the close/reap **ordering invariant** on all four exit paths with a **real Chromium child**, and it proves cancellation is honoured at the **admission gates** (post-acquire and pre-acquire refusal — 5.3). - Row 5.4 is explicit: an in-flight fetch holding the permit **runs to completion** and cancellation **at this close/reap-ordering layer** takes effect at the **next gate**. The bounded mid-flight interruption of the active CDP op is a separate path — §5b — now wired into both the PDF and HTML browser paths; its one honest residual (a deep per-socket read timeout) is in §6.2.

5b. Bounded in-flight interruption of an active browser op (real Chromium page) — both PDF and HTML paths

Boundary: `installBoundedPageInterrupt` (processor.ts), now wired into **all three** production browser call sites — `fetchPdfViaBrowser` (line ~2021), `fetchWithBrowser` (the HTML path, line ~1506), and `fetchIssuerPdfsWithPrimedSession` (line ~2298). On scope abort it drives the active page toward settlement within `BROWSER_ABORT_GRACE_MS` (500 ms) plus a gate round-trip, rather than letting the in-flight CDP op run to its own timeout. On the HTML path two supporting changes close the last fixed-wall-clock gaps: the two post-navigation settle waits are now `abortableDelay(2000, scope)` (resolve early on abort instead of always burning 2 s), and the catch block converts the puppeteer “Target closed” / “detached frame” rejection that the forced page-close produces **after** an abort into a retryable `OperationAbortedError(…, "browser_html_interrupted")`, so an interrupted HTML fetch surfaces an explicit abort rather than silently returning `""` (a dead URL).

#	Requirement	Test :: subtest	Key assertio
5b.1	A **PDF** browser op already in flight when the scope aborts is driven to settle within the grace bound, not left to its own long	`browser-inflight-interrupt.test.ts` :: B-browser (PDF)	the held page settle **the un-bounded w** once the scope abor (bounded by `BROWSER_ABORT_GRACE` gate round-trip); thr

	timeout		`OperationAbortedErr`
5b.2	An **HTML** browser op (`fetchWithBrowser`) already in flight when the scope aborts is likewise driven to settle within the grace bound and surfaces a retryable `OperationAbortedError`, not a silent `""`	`browser-inflight-interrupt.test.ts` :: B-browser (HTML path)	the held page settle `settleLatency < 300` after revoke (vs the `BROWSER_FETCH_TIMEO` `isOperationAbortedE` is true

Mutation proof (5b.1, prior session): replacing the `installBoundedPageInterrupt onAbort` body with a no-op (`MUTATION_MARKER_B3 in-flight interrupt disabled`) makes the PDF subtest **fail** — the page settles at **8169 ms** instead of inside the `< 3000 ms` bound. Reverted clean.

Mutation proof (5b.2, this session): neutralising the HTML-path wiring only (`clearInterrupt = (() => {}); /* MUTATION_MARKER … */` in `fetchWithBrowser`, leaving the PDF path untouched) makes the **HTML subtest fail at 8108 ms** (`a latency near BROWSER_FETCH_TIMEOUT=8000ms means fetchWithBrowser is not wired to the interrupt`) while the **PDF subtest still passes** — proving the new HTML wiring is independently load-bearing and is what subtest 5b.2 guards. Reverted clean; `grep -rn MUTATION_MARKER server/` → none.

Scope of this fix (honest): the bounded interrupt is now wired into **both** browser paths (PDF and HTML) and the primed-session path, so the §6.1 “HTML path not wired” gap from the prior candidate is **resolved** (see §6). What remains unproven is the deeper socket-level timeout note in §6.2 and the specific production stalls in §6.4.

---

6. Honest gaps — requirements NOT fully proven by behavioural test

These are stated so the reviewer does not read a green suite as more than it is.

1. **(RESOLVED in this candidate)** *Active in-page CDP interruption was bounded on one path only.* In the prior candidate `installBoundedPageInterrupt` was wired only into `fetchPdfViaBrowser`, leaving the `fetchWithBrowser` HTML path un-interruptible. It is **now wired into both browser paths and the primed-session path** (§5b, lines ~1506/2021/2298), proven with a real Chromium page by the new HTML subtest 5b.2 and its path-isolated mutation (725 ms wired → 8108 ms when neutralised). An HTTP fetch / `page.goto` / `page.evaluate` / WAF challenge already executing on the HTML path when the scope aborts is now driven to settle within `BROWSER_ABORT_GRACE_MS` + a gate round-trip and surfaces a retryable `OperationAbortedError`, instead of running to the browser timeout or returning a silent `""`.
2. **Deadline is now threaded into the browser fetch at the navigation + settle layers; a deep per-socket timeout is still not asserted.** Since `658a17c` the per-navigation timeouts are clamped to `remainingMs` (`gotoTimeout = base => scope ? max(1,`

`min(base, scope.remainingMs())) : base`, lines ~1496/2009), and this candidate makes the two post-navigation settle waits abort-aware via `abortableDelay` — so on both browser paths the effective ceiling on an in-flight fetch is now the scope deadline (via clamped nav timeout) or the grace bound (via the interrupt), **not** the pipeline-deadline → SIGKILL fallback. What is still **not** independently asserted by a test is a per-request TCP/socket read timeout equal to `remainingMs` inside the lowest-level puppeteer/CDP transport; the abort path tears the page down within the grace bound, which supersedes that socket in practice, but no test pins the socket layer itself. This is the honest residual of the former "bounded, not threaded" gap.

3. **(RESOLVED in the prior candidate)** The production recovery window previously degenerated to the `PDF_RECOVERY_MAX_BUDGET_MS` (600000) constant because the call site omitted `remainingMs`. The call site now feeds `remainingFetchBudgetMs(fetchPhaseStart)` — the real budget minus elapsed, clamped to a floor — so the per-company remaining budget is honoured in production, not just in the test. Proven by B1-7 and its mutation (§1). The constant is unchanged; only how the window is *derived* changed.
4. **Specific production stalls remain unobserved — the live-validation gap.** The Idemitsu ~18-minute single-step and the AXA analysis-prep ~7m26s stall (see `TIMEOUT_INVESTIGATION.md`) are **not** reproduced by any test here, and this patch has had **no live exercise**: no batch run, no paid/Railway run, no real web/LLM fetch (all forbidden by the delivery constraints). The cancellation machinery is proven at the boundaries enumerated above with real Chromium / subprocess / fork+IPC / Postgres, but whether it actually shortens those two specific production stalls can only be observed on a real run. That first real exercise is the post-deploy dashboard test, which stays with the reviewer; nothing in this document should be read as claiming those stalls are fixed.

---

7. Unattempted-URL outcomes on cancellation (explicit terminal state) — `unattempted-cancelled-boundary.test.ts`

Boundary: `markUnattemptedCancelled` (pipeline.ts), invoked at the fetch-loop cancel break. Before this candidate, the top-of-loop `if (cancelCheck?.()) break;` left still-`pending` documents silently in `pending` (the budget-exceeded path already force-marked its tail `budget_exceeded`, but the cancel path did not). The helper now walks the still-`pending` accepted documents and records each as fetch-dead with reason `not_attempted_cancelled`, giving every URL an explicit terminal outcome on cancellation.

#	Requirement	Test :: subtest	Key as
7.1	On a cancel break, every still-**pending** accepted document for the company is recorded **fetch-dead** with reason `not_attempted_cancelled` Documents already in a terminal state are **not**	`unattempted-cancelled-boundary.test.ts` :: B6-1	all 3 seeded pending `fetch_status='dead'`, `failure_reason='not_` in real Postgres

7.2	overwritten; the helper only touches `pending`	B6-2	non-pending docs ar

Mutation proof (7, this session): appending `&& false /* MUTATION_MARKER_B6 silent-drop */` to the pending filter in `markUnattemptedCancelled` makes **B6-1 fail** (`0 !== 3` — no doc marked; the silent-`pending` leak is back) while B6-2 still passes. Reverted clean.

Summary

Boundary	Behavioural test	Mutation-fail demo (this session)	Realism
B1 deadline/scope propagation + recovery window	7 subtests (B1-1..B1-7)	scope-drop → B1-1 fail; `return budgetMs` → B1-7 fail	U(broker)/U(un
B2 proc admission (in-process)	4 subtests (B2-1..B2-4)	listener-removed → B2-1 `hung`	U(broker)
B2b proc admission across real fork+IPC	1 subtest	generic-error type → withdrawal untagged, fail-fast	**I (real fork+IPC)**
B3 stalled extraction	2 subtests	abort-reg-disabled → full 60 s	**S (real kill)**
B6(sub) killable extraction	2 subtests	(subprocess boundary; see REVIEW_PACKAGE)	**S (real subprocess)**
B7 in-flight coalescing + settlement	3 subtests (B7-1..B7-3)	3 mutants → B7-1 / B7-2 / B7-3 fail	U(broker)
Browser close/reap ordering	4 paths	release-first → ordering fail (prior)	**R (real Chromium)**
Bounded in-flight page interrupt (PDF **and** HTML paths)	2 subtests (PDF 5b.1, HTML 5b.2)	PDF onAbort no-op → 8169 ms; HTML-path wiring neutralised → 8108 ms (PDF subtest still passes) — both vs < 3000 ms	**R(page) (real Chromium)**
Unattempted-URL terminal outcomes	2 subtests (B6-1, B6-2)	silent-drop → B6-1 `0 !== 3`	**D (real Postgres)**

11 test files, all green on `6e09ced`: 34 `node:test` subtests across 10 files (operation-scoped-cancellation 7, company-pdf-recovery-boundary 7, cancellable-proc-admission-boundary 4, inflight-coalescing-boundary 3, stalled-extraction-boundary 2, pdf-parse-subprocess-equivalence 2, browser-close-order 4, browser-inflight-

interrupt **2** (PDF + HTML), isolated-proc-admission-boundary 1, unattempted-cancelled-boundary 2) + `scoring-cancellation.unit` (60 passed, 0 failed; custom runner). **94 total checks, 0 failures anywhere.**

Every mutation in this document was applied, observed to fail the named assertion, and reverted to a clean tree **in this session** (`grep -rn MUTATION_MARKER server/` → none). At the candidate commit the working tree is clean (`git status --short` shows no tracked-source changes). The gaps in §6 are real and are the honest boundary of what the suite proves; item §6.4 (live validation of the Idemitsu/AXA stalls) remains the one honest gap carried forward — it requires a real run, which the delivery constraints forbid.